King County

Refactoring into a working app

Object-oriented design, tests and delivery

Roy Grossman / MLE portfolio project / October 2026

neue fische
School and Pool for Digital Talent
SPICED

What we refactored

Keep the behaviour. Give each part of the code a clear home.

STARTING POINT

Cleaning rules and geographic calculations lived inline. Reusing them meant depending on execution order and shared variables.

OUR REFACTORING PROCESS

Record behaviour → **Extract objects** → **Compare results** → **Connect the app**

Capture fixed examples | Give each one a clear job | Check values and order | Reuse the same package

Course core: reusable cleaning and feature engineering. Our extension: prediction API and Docker.

Small objects, clear responsibilities

We use composition: focused objects work together in one pipeline.

Validation	Pydantic checks incoming JSON. Our data validators check tables before transformations. Invalid inputs fail explicitly.
HouseCleaner	Apply cleaning rules to a copy. Keep validation separate and avoid changing the caller's data.
HouseFeatureEngineer	Own feature creation and fitted reference locations. Share one distance calculation instead of repeating the formula.
PredictionService	Reuse the verified pipeline. Keep API routes thin, and keep training outside the service. No extra database or training service.

Challenges and decisions

Small tasks and focused PRs helped us review one change at a time.

Safe refactoring	Regression tests compare the extracted code with fixed original results. Check feature values, column order and row alignment.
Simple OOD	Move repeated rules into the object that owns them. Keep methods small and avoid extra classes without a clear purpose.
Fast test feedback	Unit checks cover individual rules. Integration checks connect components. Contract tests protect API clients. CI jobs run in parallel.
Independent review	Challenge scope and design as we work. Audits with another model found a missing image license notice. We fixed packaging and added regression tests.

What the refactoring gave us

Reusable Python code, clear boundaries and a working local application.

A reusable package	Cleaning and feature creation work together through scikit-learn interfaces. The API calls the same code, without copying the rules.
Checks we can repeat	466 tests passed in the recorded final local run. Ruff, package builds and separate CI test jobs support review before merging.
A packaged application	Docker runs the API as a non-root user. The model stays on a read-only mount, and its approved checksum gates loading.

Main lesson: refactoring makes behaviour easier to understand, reuse and verify. Passing software tests does not prove prediction quality.

Still pending: corrected public image release and redistribution checks. No public API deployment.

github.com/roy-nfisch/mle-refactoring-project